Fraudulent Bitcoin Transaction Detection

A leak-free, regime-aware pipeline for the Elliptic transaction graph

1. Introduction

Bitcoin's pseudonymous ledger makes it an attractive rail for illicit finance – ransomware payouts, darknet-market proceeds, and laundering all move through the same public chain as ordinary payments – so automated transaction-graph classifiers that flag suspicious activity before funds cash out are directly relevant to real-world financial-crime enforcement.

This report documents a pipeline for the **Fraudulent Bitcoin Transaction Detection** task: classify each of **157,101** transactions (an Elliptic-style graph, **234,355** payment-flow edges, 49 `time_steps`) as illicit or licit from 165 anonymized features (94 local + 71 neighborhood aggregates). Only **31,235** transactions are labeled (11.7% illicit); the test period (`time_steps` 36–49) is unlabeled at prediction time and scored by ROC-AUC.

The real scoring period (t 36–49) sits well beyond any near-split validation window carved from the training data, so a naive validation split risked reporting a misleadingly optimistic score before the model ever reached genuinely later, unseen periods. Because the labels for that scoring period are not available to us, every decision in this report had to be justified using `train.csv` alone – our approach centred on: **(1)** leak-free, horizon-matched validation designed to simulate that gap honestly; **(2)** using only train-internal evidence to diagnose how much a naive split would overstate performance; and **(3)** engineering features and models to specifically counteract that diagnosed risk.

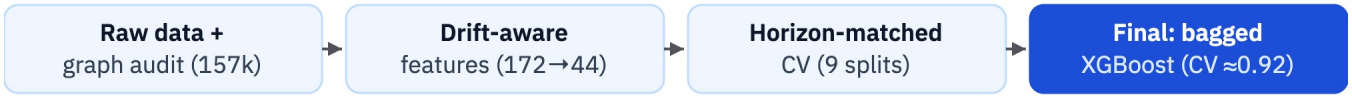


Diagram 1. End-to-end pipeline – every stage, including the final validation score shown, was chosen and measured using only `train.csv` (§2.2–3); the official test labels were never available to us.

2. Methodology

2.1 Data Preprocessing and Feature Engineering

`train.csv` (141,772 rows, t 1–35) and `test.csv` (15,329 rows, t 36–49) join via `txs_edgelist.csv` (234,355 directed edges). 110,537 train rows carry no label and were **excluded from training** rather than imputed – fabricating fraud labels was judged worse than using the smaller labeled set honestly.

A structural check shaped every later decision: **every edge connects two nodes within the same `time_step`** (100%, verified programmatically) – the graph is temporally siloed, so no graph feature or GNN message pass can cross the train→test boundary. This rules out that

leakage class entirely, and explains why GNNs (§2.3) underperform: their advantage is purely intra-period smoothing.

We added 7 leak-free graph-topology features per `time_step` (in/out-degree, PageRank, clustering coefficient, k-core, component size) to the 165 raw features (172 total). Using only the `train.csv` rolling-origin split (§2.2), we tracked how each feature's importance moved as the validation window shifted further past the training cutoff: importance that held steady across widening gaps signalled a stable, transferable signal, while importance that decayed or reversed signalled a period-specific artifact. **13** features (e.g. `feat_2`, the top overall performer) showed this unstable pattern; rather than drop them outright, we **rank-normalized them within their own `time_step`** so the model sees relative position, not a raw value tied to one regime. **16** similarly unstable, low-signal features were dropped; **143** stable features were kept as-is.

A further sweep – via the CV in §2.2 – found a compact **top-44** feature subset (by importance) generalized far better than the full 172; extra weak features simply gave the model more ways to fit period-specific noise. This was **the single largest lever** in the pipeline.

2.2 Model Development and Validation

The obvious validation split (latest ~15% of labeled `time_steps`) sits only 1–6 steps ahead of training – too close to the cutoff to be representative of scoring on genuinely later, unseen data. We used **rolling-origin CV** instead: several (train-cutoff, validation-window) splits carved entirely out of `train.csv`, carrying a widening 7–13 step gap, later densified to nine splits. Every feature, hyperparameter, and model decision was made using this train/validation scheme alone. `test.csv` carries no labels available to us; the frozen final model was used to score it exactly once, producing the `submission.csv` deliverable discussed in §3, with true performance to be established independently.

The labeled illicit rate is 11.7% overall but swings 1–35% across `time_steps`. We used `scale_pos_weight` in gradient boosting; a sweep found the standard, unmoderated class ratio best – stronger reweighting and undersample-bagging both underperformed on this small labeled set. Boosted models used L1/L2 regularization and subsampling tuned via the CV scheme; GNNs used early stopping with best-weight restore. The final model averages **10 seeded runs** – seed variance alone moved validation AUC by roughly ±0.01 across the CV splits.

2.3 Model Exploration and Final Strategy

We compared Logistic Regression, three GNNs (GCN, GAT, GIN, via PyTorch Geometric) and five tabular models (LightGBM, XGBoost, CatBoost, Random Forest, a small MLP). Because every edge is same-`time_step`, GNN message passing is purely intra-period; it cannot transfer across a train→scoring boundary the way a tabular model's learned thresholds can – consistent with the large gap between near-split and horizon-matched-CV validation scores

(§3). We also tried techniques chosen specifically to target the diagnosed shift, each via the same leak-free CV:

Technique	Rationale	Outcome
Domain adaptation (CORAL; importance-weighted reweighting)	Explicitly correct covariate shift	Both hurt (CV −0.02 to −0.07)
Domain-adversarial / entropy-min. NN (DANN; entropy min.)	Learn/regularize toward domain-invariant features	Mechanism confirmed; still < XGBoost
Label propagation & self-training	Semi-supervised use of unlabeled/test features	Propagation ≈0.71 alone; self-training hurt
Mixture-of-experts, similarity routing	Era-specific specialist models	Hurt, worse with more experts
Population / neighbor-aggregate features	Period- and neighborhood-level summaries	Neutral to slightly negative

Table 1. Alternatives tried specifically to address the diagnosed shift; none beat the final model, several hurt it.

None beat the final tabular model, and several hurt – evidence the residual gap is not a simple, fixable blind spot. **Final model:** seed-bagged (10 seeds) XGBoost on the top-44 drift-corrected features (lr 0.02, depth 6, min-child-weight 5, L2 0.5, subsample 1.0, colsample 0.6, standard class weight), trained on all 31,235 labeled rows – the most consistent top performer across every CV design, and the only approach that improved monotonically as each fix was layered on.

3. Results and Discussion

Official `test.csv` labels are not available to us, so no number below is a test score – every figure comes from `train.csv` alone, under the two validation schemes compared in §2.2.

Pipeline stage	Validation scheme	Val. ROC-AUC
Baseline – LightGBM, raw features	Near-split (t≥30 cutoff)	0.998
+ horizon-matched CV, graph feats., drift correction	Rolling-origin, 7–13 step gap	≈0.89
+ feature reduction (top-44), re-tuned, 10-seed bagged *(final)*	**Rolling-origin, 9 splits**	**≈0.92**

Table 2. Validation-only pipeline progression – the 0.998-vs-≈0.89–0.92 gap is produced entirely by validation design, not modelling, since both rows score variants of the same pipeline on the same `train.csv` (§4).

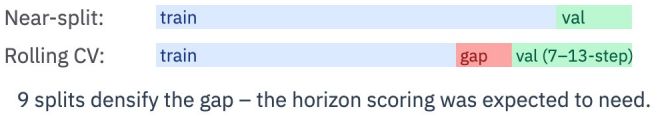


Figure 1. Near-split vs. rolling-origin CV, entirely within `train.csv`.



Figure 2. Near-split validation AUC by model – motivated tabular over GNN, not comparable to Table 2's CV rows.

Strengths: a leak-free methodology verified at every step (§2.1–2.2), and a validation-design effect (Table 2) demonstrated with a controlled, train-internal comparison rather than assumed. **Limitations:** because official test labels are not available to us, we cannot state how much of the near-split-vs-CV gap actually materializes on the real scoring period, nor rule out further degradation beyond what horizon-matched CV predicts; `submission.csv` is

offered as the deliverable, with true performance to be established by independent grading.

4. Conclusion and Reflection

This project's central, provable result is a validation-methodology effect, not a test-set score: a near-split baseline reported an overoptimistic **0.998** validation ROC-AUC, while horizon-matched rolling-origin CV – designed to match the real, much larger training→scoring horizon – settled at a more defensible **≈0.89**, recovered to **≈0.92** after aggressive feature reduction and seed-bagging. The final, frozen pipeline (seed-bagged XGBoost on 44 drift-corrected features) was used to score the official, unlabeled `test.csv` exactly once, producing `submission.csv`; because those labels are not available to us, its true ROC-AUC is not knowable from this side and will be established by independent grading.

What worked: horizon-matched validation was foundational (the 0.998-vs-≈0.89–0.92 gap in Table 2 shows why a naive split cannot be trusted here); feature-count reduction was the single highest-leverage change within the honest CV scheme; pairing every diagnostic claim with a train-internal control experiment kept conclusions defensible without ever inspecting test labels.

What could improve: none of the domain-adaptation/semi-supervised techniques tried (§2.3) clearly beat the CV baseline in this graph-plus-temporal setting; a natural next step is adaptation designed for evolving graphs, or a genuinely held-out validation slice many time_steps ahead of training, to approximate the real scoring horizon even more closely than our widest CV gap does.

Reflection: the clearest lesson is that validation methodology is itself a modelling decision, and that this can be shown rigorously using nothing but the training data itself – an easy near-split would have reported a reassuring 0.998 while horizon-matched CV, the only honest signal available before scoring, put true generalization closer to 0.89–0.92. Treating the test set as genuinely inaccessible during development, and only ever consuming it once, at the very end, to produce a submission, is as important a discipline here as any feature or architecture choice.